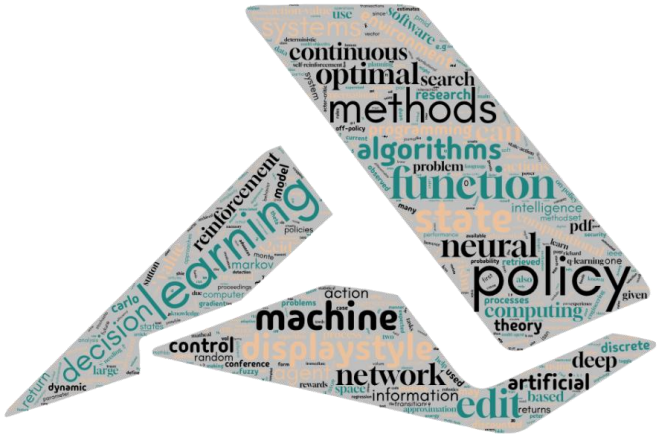


Reinforcement Learning Algorithms and Applications

Chapter 4: Q-Learning for Gridworld

Introduction and Setup

- **Q-Learning Agent** implementation for navigating a 4x4 gridworld environment.
- Agent must learn to reach the **goal** while avoiding **pits** and **walls**.
- Uses **epsilon-greedy strategy** for balancing exploration and exploitation.
- Environment provides **rewards**: +10 for goal, -10 for pit, -1 for each step.
- **State space**: 16 positions (4x4 grid), **Action space**: 4 directions (up, down, left, right).
- **Gridworld wrapper** provides standard RL interface for the environment.
- Training involves **1000 episodes** with maximum 200 steps per episode.
- P: Starting position of the agent
- +: Goal state (the agent tries to reach this)
- −: Trap state (the agent should avoid this)
- W: Wall or obstacle that blocks movement

```
[['+' '-' ' ' 'P']
 [' ' 'W' ' ' ' ']
 [' ' ' ' ' ' ' ']
 [' ' ' ' ' ' ' ']]
```

Q-Learning Algorithm Parameters

- **Learning rate (α)**: 0.1 - controls how much new information overrides old information.
- **Discount factor (γ)**: 0.95 - determines importance of future rewards.
- **Epsilon**: 1.0 initially - probability of taking random action for exploration.
- **Epsilon decay**: 0.995 - gradual reduction of exploration over time.
- **Epsilon minimum**: 0.01 - maintains small amount of exploration throughout training.
- **Q-table initialization**: zeros matrix with dimensions (state_size × action_size).

Q-Learning Update Rule

- **Q-Learning** is a value-based reinforcement learning algorithm.
- It aims to learn the **Q-function** $Q(s,a)$,which represents the expected cumulative reward of taking action a in state s.
- **Update Rule**
  - $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$
- **Where:**
  - $Q(s,a)$: current Q-value for taking action a in state s
  - α: learning rate
  - r: immediate reward received after taking action a
  - γ: discount factor
  - s': next state
  - $\max_{a'} Q(s',a')$: maximum estimated Q-value in the next state
  - Target Q-value: $r + \gamma \max_{a'} Q(s',a')$

Agent Architecture and Methods

- **get_action()** method implements epsilon-greedy action selection strategy.
- **update_q_table()** applies Q-learning update rule after each action.
- **decay_epsilon()** reduces exploration rate over training episodes.
- **State representation**: converts 2D grid position to 1D state index.
- **Action mapping**: 0=up, 1=down, 2=left, 3=right for consistent indexing.
- **Terminal state handling**: Q-value update considers episode termination.
- **Q-table** stores learned values for all state-action pairs.

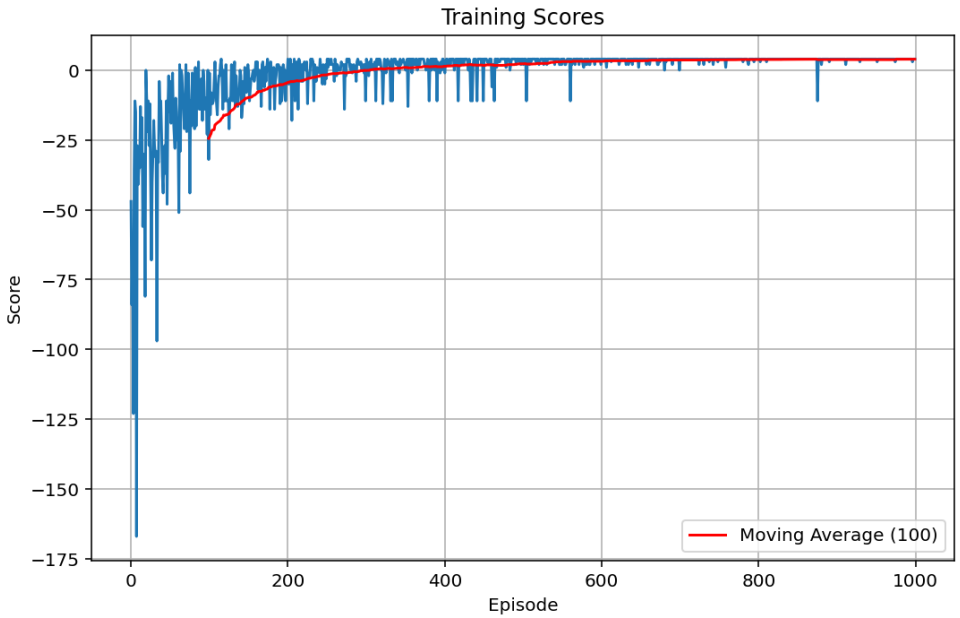
Training Process and Results

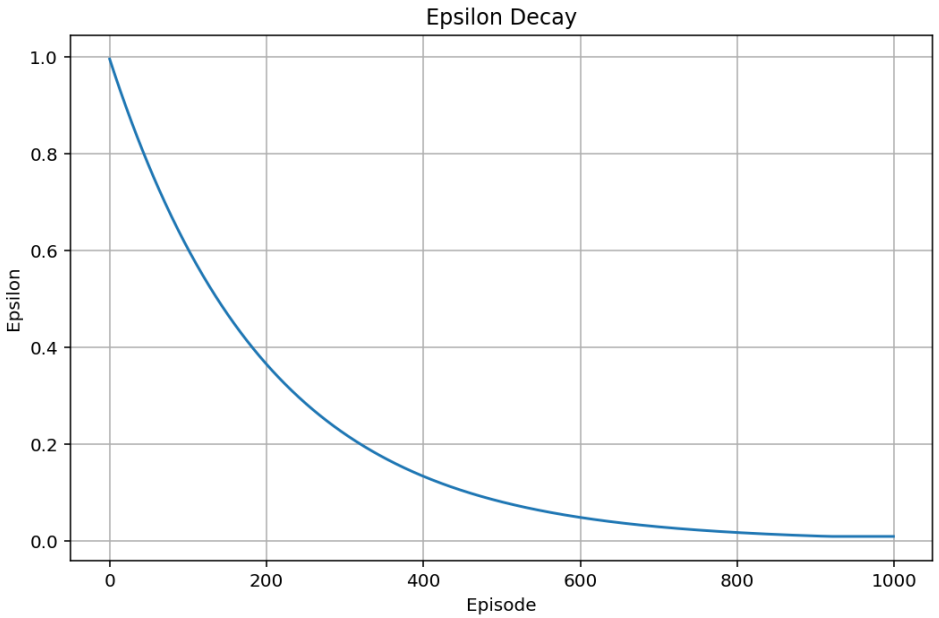
- **Training loop** executes 1000 episodes with progress tracking every 100 episodes.
- **Score tracking** monitors total reward per episode for performance evaluation.
- **Epsilon tracking** shows exploration rate decay throughout training.
- **Moving average** calculation (100-episode window) for smoothed performance trends.
- **Final training metrics**: average score of last 100 episodes and final epsilon value.
- **Convergence assessment** based on score stability and policy consistency.
- **Environment analysis** displays positions of player, goal, pit, and wall components.

Visualization and Analysis

- **Training curves** plot episode scores and epsilon decay over time.
- **Q-table display** shows learned Q-values for all state-action combinations.
- **Policy visualization** uses directional arrows to show optimal actions per state.
- **State values** display maximum Q-value for each grid position.
- **Performance metrics** include average scores and convergence indicators.
- **Moving average smoothing** reduces noise in training curve visualization.
- **Grid-based policy representation** provides intuitive understanding of learned behavior.

Visualization and Analysis





Testing and Demonstration

- **Test phase** runs 10 episodes with **epsilon=0** (pure exploitation).
- **Path tracking** records complete sequence of states visited during testing.
- **Step-by-step demonstration** shows Q-values and action selection process.
- **Performance evaluation** compares test scores with training performance.
- **Success criteria**: positive total reward indicates reaching the goal.
- **Complete walkthrough** displays the full journey from start to goal.
- **Final assessment** determines if Q-learning successfully learned optimal policy.

Key Findings and Conclusions

- **Q-learning effectiveness** demonstrated through successful navigation to goal.
- **Exploration-exploitation balance** achieved through epsilon-greedy strategy and decay.
- **Policy convergence** evidenced by consistent high performance in test episodes.
- **State value distribution** shows higher values near goal and lower near obstacles.
- **Algorithm robustness** confirmed through multiple test runs and consistent results.
- **Practical implications** for reinforcement learning in discrete state spaces.
- **Future improvements** could include function approximation for larger state spaces.

Reference

- M. Lapan, *Deep Reinforcement Learning Hands-On*, 2nd ed. Birmingham, UK: Packt Publishing, 2020.
- A. Zai and B. Brown, *Deep Reinforcement Learning in Action*. Shelter Island, NY: Manning Publications, 2020.
- S. Ravichandiran, *Deep Reinforcement Learning with Python*, 2nd ed. Birmingham, UK: Packt Publishing, 2020.